Architecting a Scalable Live Streaming Platform

At its core, a live streaming platform enables creators to broadcast video content while allowing viewers to consume and interact with that content in real time. Modern examples like Twitch, Kick, and various sports streaming services demonstrate the complexity of balancing high availability, low latency, and massive scale. This post outlines a robust system design for such a platform.

Functional Requirements

To design an effective platform, we must first establish the essential functional requirements:

- Users should be able to live stream video on the platform.
- Users should be able to watch live stream video on the platform.
- Users should be able to watch videos on demand on the platform.
- Users should be able to interact while watching the video.

Non-Functional Requirements

To provide a high-quality user experience, the system must satisfy the following non-functional requirements:

- The system should be highly available (prioritizing availability over consistency).
- The system must guarantee low-latency video streaming(> 5s).
- The system must be scalable.

Scale

To establish a comprehensive understanding of the system requirements, it is essential to define its operational scale. The system is engineered to maintain a baseline capability that addresses the following performance metrics seamlessly:

- A peak global concurrency of 1,000,000 Concurrent Viewers (CCU).
- Support for 10,000 concurrent streamers.
- An inbound chat throughput of 50,000 published messages per second.
- An outbound message fanout delivery rate of 5,000,000 messages per second across 100,000 active chatters in a high-traffic channel.

Core Entities

Clarifying the core entities within the system is essential for guiding the architectural design:

- **User**: Individuals participating in the platform as either content creators (streamers) or consumers (viewers).
- **Video**: The primary asset of the system, which may be a live broadcast or a persisted video-on-demand (VOD) file.

- **Video Metadata**: Descriptive data including stream status, creator information, and technical specifications used for discovery and management.

API

The API serves as the primary interface for user interaction. Defining clear endpoints helps structure the high-level design and ensures all functional requirements are met:

- **View Live Stream**: `GET /v1/videos/{video_id}`
- **Upload Video**: `POST /v1/videos/upload`
- **Post Comment (Live Chat)**: `POST /v1/videos/{video_id}/comments`
- **Get Video Chunk (Video on Demand)**: `GET /v1/videos/{video_id}/chunks/{chunk_name}`

Note: These HTTP endpoints represent high-level abstractions. Specialized protocols (e.g., RTMP, WebSockets) will be utilized for the actual data transmission layers discussed later.

High-Level Design of the System

An efficient live streaming system requires a symphony of components: an API control plane, an ingestion pipeline for video processing, specialized databases for metadata and chat, a Content Delivery Network (CDN) for distribution, and a pub/sub broker for real-time messaging.

Components:

1. User / Viewer

As previously defined (streamer or viewer).

2. API Server

Serves as the control plane of the system and an authenticator for users, whether they are streamers or viewers.

- **Why API Server?**
  - Handles security and control operations.
  - Abstracts other internal services.
- **Scaling Strategy:**
  - Addresses high concurrent traffic from users for authentication and control signals.
  - Can be scaled horizontally by deploying API pods behind a load balancer.

3. Ingestion Pipeline

The ingestion pipeline transcodes incoming streams into various bitrates using an Adaptive Bitrate (ABR) ladder. It utilizes a frame buffer to hold transcoded data until it can be packaged into discrete video chunks by the packager service.

- **Purpose**: Enables ABR for diverse viewer network conditions and packages raw video into consumable segments.
- **Scaling Strategy:**
    - **Transcoder Service**: A computationally heavy process hidden behind a GPU cluster that can scale dynamically.
    - **Buffer Management**: Prevents memory exhaustion during scaling by using Time-To-Live (TTL) eviction.
    - **Packager Service**: Kept stateless, utilizing horizontal auto-scaling for packaging workers.

4. Video Storage

We utilize a distributed object storage service (e.g., AWS S3) for long-term video archiving.

- Eliminates physical hardware constraints through elastic scaling.
- Avoids the limitations of traditional physical hard drives.
- Scales practically infinitely.
- Highly cost-efficient (offers different storage tiers).
- Provides high read/write throughput.

5. Video Metadata Database

Uses a PostgreSQL database to store core video metadata.

6. Chat Database

A NoSQL database like Apache Cassandra is used to handle the high-velocity write traffic of real-time chat.

- Optimized for fast writes and linear scalability to handle millions of simultaneous messages.

7. WebSocket Server and Gateway

Serves as the entry point for users connecting to the chat service.

- **Why WebSocket Server & Gateway?**
    - Manages connections for all users wanting to use the chat service.
    - Routes messages to the correct channels.

.

8. Pub/Sub Broker

A distributed streaming platform (e.g., Apache Kafka) manages the message distribution between chat producers and consumers.

- **Purpose**: Decouples the WebSocket gateway from database persistence and enables asynchronous message broadcasting.

9. Content Delivery Network (CDN) and Origin Shield

The CDN is used for the global distribution of video chunks, while the Origin Shield protects the primary storage from traffic surges.

- **Purpose**: Reduces latency by caching content closer to viewers and prevents origin overload during high-traffic events.

10. Network Protocols

How components communicate:

- **User to API Server**: HTTP
- **User to Chat Service**: WebSocket
- **User to Ingestion Pipeline**: RTMP
- **User to CDN**: LL-HLS

System Data Flows and Sequence Diagrams

The efficacy of a system is defined not merely by its constituent components, but by the orchestration of their interactions and the underlying data flows. To satisfy the previously established functional requirements, we must analyze the specific mechanics of video ingestion, viewer playback, and real-time user interaction within the platform.

1. Live Stream Ingestion Flow

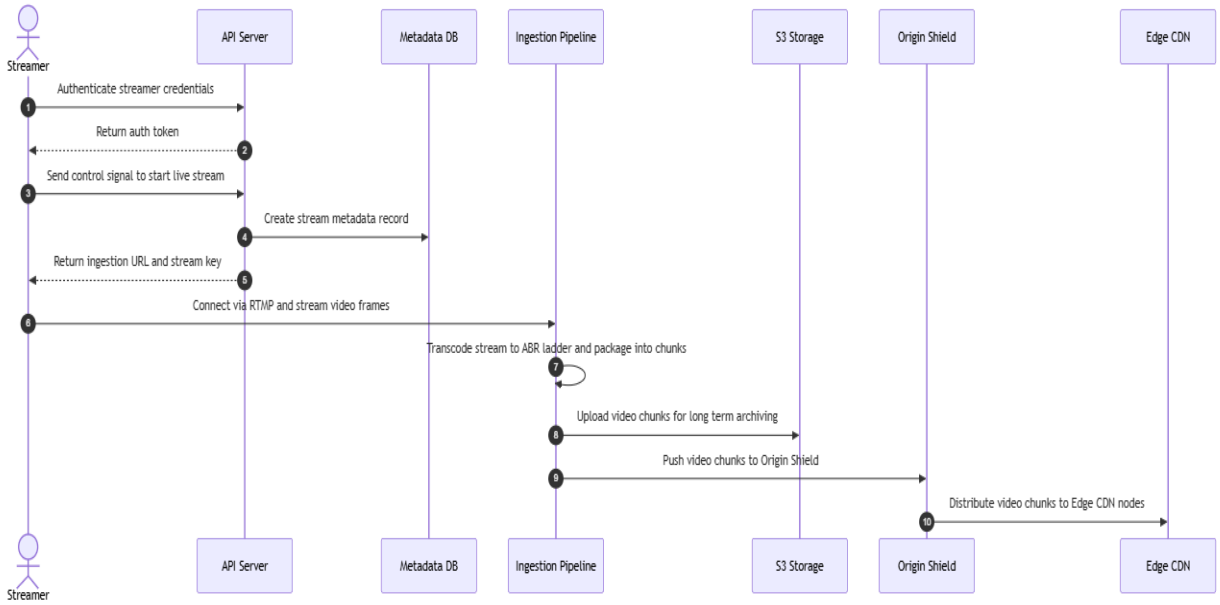

Figure 1: Live Stream Ingestion Flow

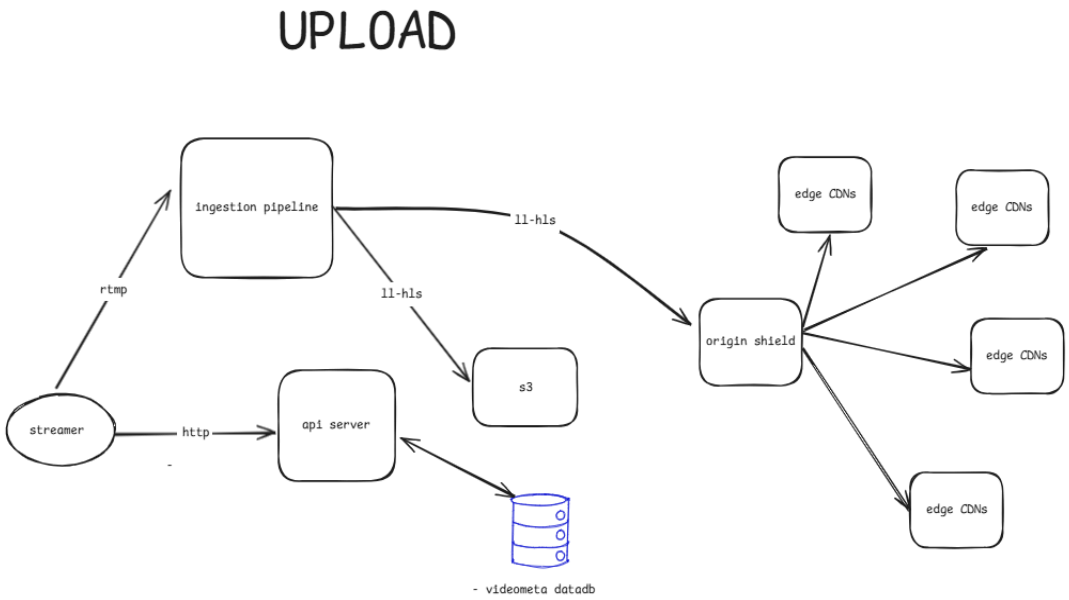

Figure 2: Live Stream Ingestion Architecture

Ingestion Flow Analysis (as shown in Figure 1 and Figure 2)

1. The streamer sends an authentication request with credentials to the API Server.
2. The API Server verifies the credentials and returns an authentication token to the streamer.
3. The streamer sends a control signal request to the API Server to initiate a new live stream.
4. The API Server writes the initial video metadata record (with status set to live) into the Metadata Database.

5. The API Server returns the RTMP ingestion server URL and unique stream key to the streamer.
6. The streamer connects to the Ingestion Pipeline using RTMP protocol and continuously uploads raw video frames.
7. The Ingestion Pipeline transcodes video frames into an Adaptive Bitrate (ABR) ladder for multiple resolutions, buffers the frames, and packages them into discrete video chunks.
8. The Ingestion Pipeline asynchronously uploads the video chunks to S3 Storage for permanent archiving.
9. The Ingestion Pipeline pushes the live video chunks to the Origin Shield.
10. The Origin Shield distributes the video chunks to Edge CDN nodes for viewer delivery.

2. Viewer Playback Flow

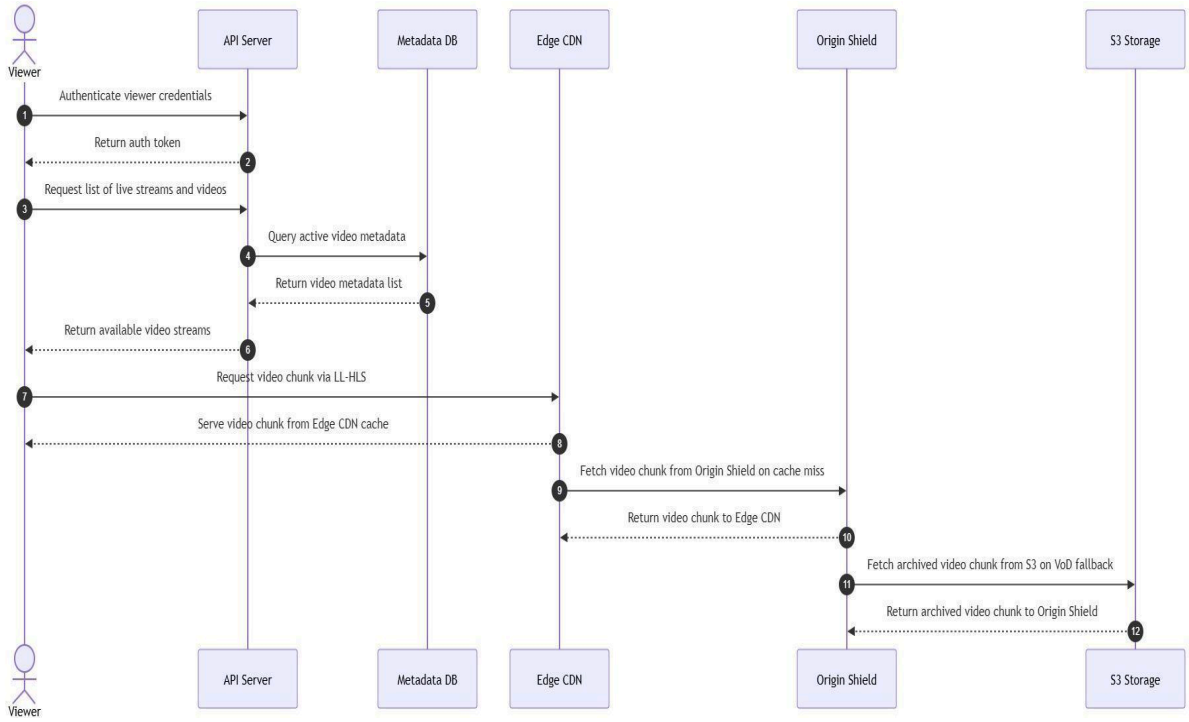

Figure 3: Viewer Playback Flow

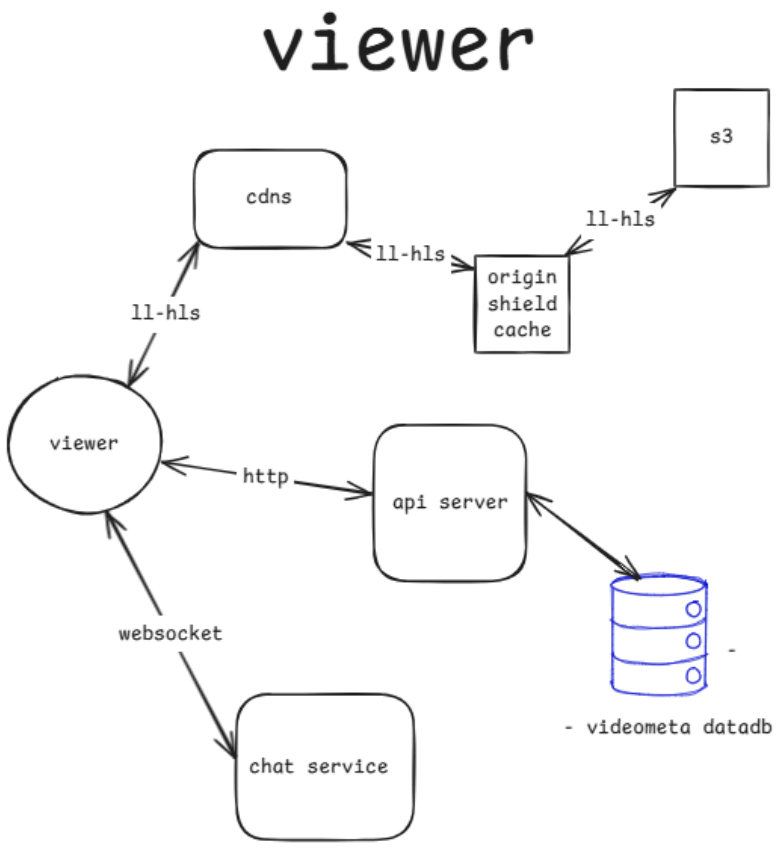

Figure 4: Viewer Playback Architecture

Viewer Playback Flow Analysis (as shown in Figure 3 and Figure 4)

1. The viewer sends an authentication request with credentials to the API Server.
2. The API Server verifies the credentials and returns an authentication token to the viewer.
3. The viewer sends a request to the API Server to get a list of recommended live streams and video-on-demand content.
4. The API Server queries the Metadata Database for active live streams and video records.
5. The Metadata Database returns the matching video metadata records to the API Server.
6. The API Server sends the list of available streams and metadata to the viewer.
7. The viewer requests a specific live stream video chunk from the Edge CDN using an LL-HLS connection.
8. If available, the Edge CDN immediately serves the requested video chunk from its local cache to the viewer.
9. If the chunk is missing from the Edge CDN cache, the Edge CDN requests the video chunk from the Origin Shield.
10. The Origin Shield returns the video chunk to the Edge CDN, which caches it and serves it to the viewer.

11. If a video-on-demand chunk is missing from the Origin Shield, the Origin Shield fetches the archived chunk from S3 Storage.
12. S3 Storage returns the archived video chunk to the Origin Shield to be cached and served down to the Edge CDN and viewer.

3. Real-Time Chat Service Flow

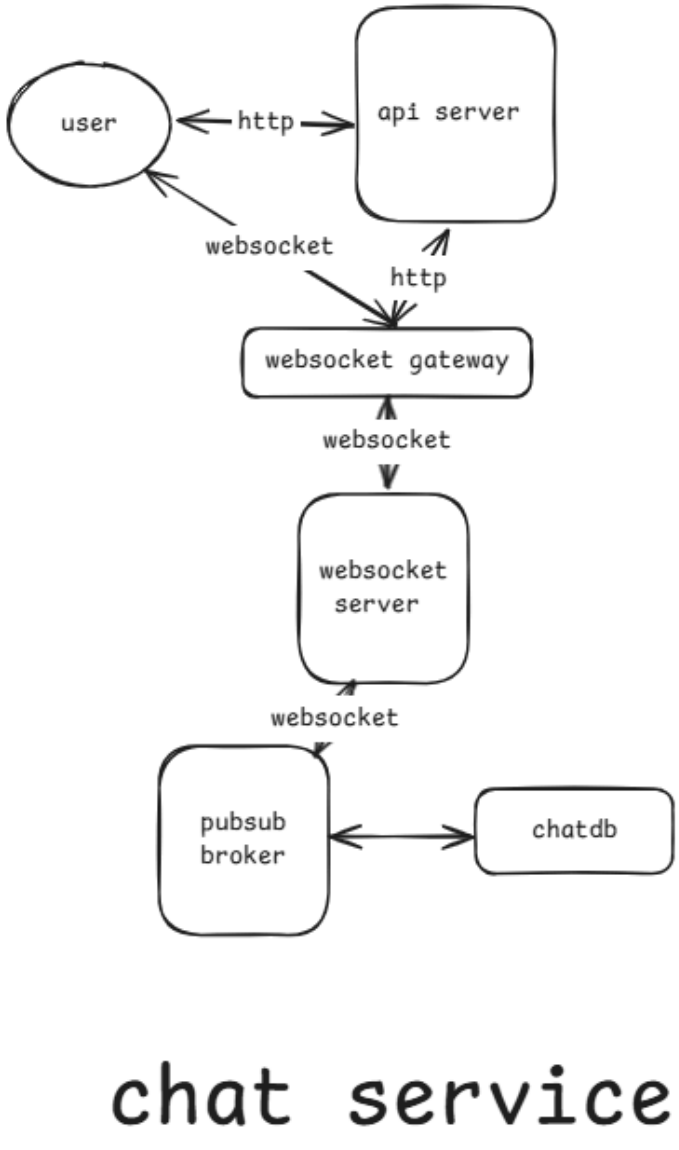

Figure 5: Real-Time Chat Service Architecture

Real-Time Chat Flow Analysis (as shown in Figure 5)

1. The user (streamer or viewer) authenticates with the API Server to obtain an authentication token.
2. The API Server validates the user credentials and returns an authentication token.
3. The user initiates a WebSocket connection to the WebSocket Gateway using the authentication token.
4. The WebSocket Gateway verifies the authentication token with the API Server.
5. The API Server confirms token validity and user authorization details to the WebSocket Gateway.
6. The WebSocket Gateway accepts the connection and establishes a persistent WebSocket session with the user.
7. The user sends a real-time chat message frame to the WebSocket Gateway.
8. The WebSocket Gateway publishes the chat message to the corresponding channel topic in the Kafka Pub/Sub Broker.
9. The Kafka Pub/Sub Broker asynchronously writes the chat message to the Cassandra Chat Database for persistence.
10. The Kafka Pub/Sub Broker broadcasts the chat message to all WebSocket Gateway nodes subscribed to that stream channel.
11. The WebSocket Gateways deliver the real-time chat message frame to all connected viewers watching the stream.

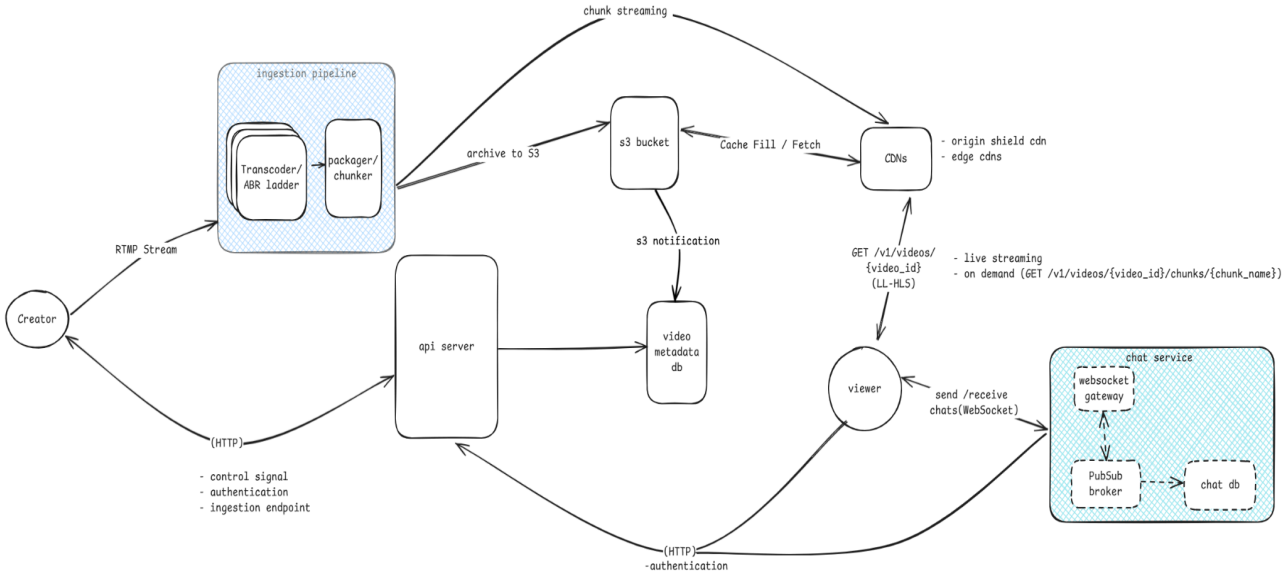

Figure 6: High-Level System Architecture

Final overview design of the system, as shown in Figure 6.

Deep Dive: Architectural Trade-offs and Scalability

In this section, we analyze how our architecture addresses the core non-functional requirements through the lens of the CAP theorem and distributed systems best practices.

1. CAP Theorem: Availability vs. Consistency

The CAP theorem states that a distributed system can only provide two of three guarantees: Consistency, Availability, and Partition Tolerance. For a scalable live streaming platform, real-world networks always face connection faults, making partition tolerance non-negotiable. Therefore, we must strategically choose between availability (AP) and consistency (CP) across different subsystems:

- **Video Delivery (AP)**: Prioritizes availability and partition tolerance. When a network partition occurs between components, the viewer must still be able to watch the stream without playback buffering or a system crash, even if it means edge nodes temporarily receive chunks out of order or experience a delayed update in video metadata. Eventual consistency is perfectly acceptable here. For example, in Low-Latency HLS (LL-HLS), viewers in different geographic regions might experience a 1–2 second playback drift, but they continue receiving video segments uninterrupted. Similarly, in VOD playback, if an updated video title hasn't propagated worldwide, the viewer still watches the video using stale cached metadata rather than receiving an HTTP 500 error.

- **Control Plane (CP)**: Prioritizes strong consistency. Core operations like stream key authentication, user credentials, subscriber payments, and single-use signed playback tokens require linearizable consistency. The system cannot allow an invalid or revoked token to authenticate simply because a database node hasn't synced yet. If a network partition occurs, the API server will explicitly sacrifice availability, rejecting incoming write requests until consensus is restored using standard agreement methods, prioritizing data correctness over absolute uptime.

- **Chat Service (AP)**: Optimized fully for availability. During massive chat spikes (e.g., 500,000 users concurrently in a single channel), the PubSub brokers and WebSocket gateways prioritize broadcasting messages in real time over guaranteeing strict chronological message ordering for every single user. If a network partition isolates a WebSocket worker, some chat messages might arrive slightly out of sequence or be dropped. However, as messages are asynchronously written from the PubSub engine into the chat database, the store eventually catches up to maintain consistent historical chat logs.

2. Low-Latency Optimization Techniques

Achieving minimal glass-to-glass latency within a live video streaming infrastructure necessitates a series of synchronized optimizations across every architectural layer. The following breakdown details the specific mechanisms utilized to control these latencies:

I. Ingestion Layer: High-Performance Protocols and Ingest

At the point of ingestion, the system utilizes the RTMP protocol to facilitate persistent connections for streamers, bypassing the inherent overhead of traditional file-based transfers. Video processing occurs entirely within dedicated GPU memory, eliminating disk I/O bottlenecks. By maintaining data within high-speed memory during the transcoding phase, the system achieves sub-100ms preparation times for incoming streams.

II. Packaging Layer: Low-Latency HLS (LL-HLS) Segmenting

The packaging stage leverages Low-Latency HLS (LL-HLS) to minimize delivery delay. Unlike legacy protocols that require full segment completion, which often exceeds 6 seconds, the system decomposes segments into microscopic "parts" with fractional-second durations. These parts are broadcast immediately via chunked transfer, allowing viewers to begin playback while the remainder of the segment is still being processed.

III. Edge Delivery Layer: In-Memory Origin Shielding and Caching

To optimize delivery, the architecture implements an "Origin Shield," which is a high-performance, memory-resident cache that protects primary storage from request surges. This layer collapses redundant chunk requests from global edge nodes, ensuring that high-traffic events do not saturate the origin. Distributed CDN nodes pull directly from this hot memory path to maintain rapid content propagation.

IV. Viewer Side: Client-Driven Adaptive Bitrate (ABR)

On the consumption end, the player is engineered for resilience against network fluctuations. If playback drift occurs due to network congestion, the client automatically applies a slight speed increase (e.g., 1.05x) to synchronize with the live edge. For severe throughput drops, the ABR algorithm initiates an immediate bitrate downshift to prevent buffering, while aggressive pre-roll logic allows for near-instant playback start.

3. Designing for Massive Scale

To support our baseline targets of 1,000,000 CCU, 10,000 concurrent streamers, and a chat throughput of 50,000 messages per second, we implement the following horizontal scaling strategies:

I. Processing Video Streams (Transcoding)

To support 10,000 concurrent streamers, the system must overcome the primary bottleneck at the transcoding layer. We employ the following strategies to maintain performance:

- **Elastic GPU Clusters**: Transcoding nodes are deployed within auto-scaling groups. As streamer concurrency increases, the cluster scales dynamically using Kubernetes clusters pinned to dedicated hardware GPU instances (e.g., AWS A10G/T4).
- **Workload Distribution and Partitioning**: Incoming RTMP streams are load-balanced across the cluster. To ensure efficiency, we assign streams to isolated GPU instances via stream key hashes, preventing single-node bottlenecks.
- **Buffer Management**: We utilize RabbitMQ and Kafka to buffer transcoded frame segments. This decouples the ingestion pipeline from the packaging service, ensuring that spikes in frame generation do not lock packaging operations or cause memory overflow.

II. Packaging for Delivery

To manage the massive scale of generating millions of video chunks without encountering I/O bottlenecks or storage write latency, we employ a stateless, memory-first architecture.

- **Stateless Packager Workers:** Our packager services are designed to be stateless, allowing them to scale horizontally using Kubernetes Horizontal Pod Autoscalers (HPA). These workers scale dynamically based on real-time queue depth from the ingestion frame buffer, ensuring consistent performance regardless of incoming stream volume.
- **Dual-Path Routing:** To prevent storage write lag, packagers bypass physical disks entirely during the live transmission phase. Live edge chunks are pushed directly in-memory to the Origin Shield using HTTP POST requests. Simultaneously, these chunks are asynchronously queued for long-term archiving to S3, using multi-part in-memory byte streams to ensure the live path remains unblocked.
- **Memory-First Architecture:** By keeping the "hot" path purely memory-resident, we eliminate the latency associated with disk access. Only fully completed segments are offloaded to persistent S3 storage, while the Origin Shield provides an immediate, high-speed cache for concurrent viewer requests. This decoupling ensures that generating millions of chunks does not overwhelm the core storage layer.

III. Viewer Traffic and Distribution Optimization

Managing a global peak of 1,000,000 concurrent viewers accessing live streams or VOD assets presents a significant distribution challenge that necessitates robust shielding and caching strategies.

- **Multi-Layer Caching**: The architecture implements an Origin Shield layer positioned between the S3 storage and the CDNs. This cache effectively collapses redundant requests for identical video chunks, protecting the primary storage from thundering herd scenarios.

- **Global CDN**: Content is edge-cached across a global network to minimize latency. This ensures that massive viewer volumes are served from geographically proximal nodes rather than saturating the origin infrastructure.

- **Multi-Tier Origin Shield Caching:** A high-performance in-memory cache layer, utilizing tools like Redis, sits in front of the packager and S3. This layer is engineered to absorb up to 99% of incoming chunk requests.
- **Global Edge CDN Offloading:** Viewers retrieve media segments directly from distributed Edge CDNs. These nodes cache segments locally, allowing the core system to serve only a single chunk per region regardless of the local viewer density.

IV. The API Server

To support millions of active participants, the control plane must be engineered for extreme resilience, ensuring that user activities are managed through a system that remains stable under immense pressure.
The system is designed to handle massive traffic surges, particularly "Going Live" spikes where thousands of viewers attempt to join a single stream within a few seconds.

- **Stateless API Services:** API nodes are designed to be stateless, facilitating horizontal scaling behind Layer-7 Load Balancers to manage control signal volume.
- **Heavy Caching Layer:** Read-heavy metadata, including creator profiles and live statuses, is persisted in Redis cluster layers. This provides sub-millisecond lookups and reduces direct database read queries to negligible levels.

V. Chat Infrastructure

To effectively scale the chat ecosystem, we must architect the infrastructure as a multi-layered component. This requires optimizing the connection layer for WebSocket management, the fanout layer for robust pub/sub brokering, the persistence layer for high-velocity storage, and the client-side interface to ensure fluid message delivery.

A. The Connection Layer (WebSocket Gateways)

Managing 100,000 concurrent connections on a single instance is impractical. Instead, we utilize a horizontally scaled fleet of WebSocket Gateway pods to distribute the load.

- **Stateless Gateways**: These servers are dedicated to maintaining persistent TCP/WebSocket connections and streaming raw data to clients, abstracting away business logic, persistence operations, and authentication.
- **Connection Capacity:** A modernized, optimized server typically handles approximately 50,000 active WebSocket sessions. Supporting millions of participants requires deploying hundreds of these lightweight pods behind a high-performance load balancer.
- **Connection Registry:** An in-memory store, such as Redis, facilitates rapid mapping of user_id to specific gateway_server_id records, ensuring the system accurately identifies the host node for every user.

B. The Fanout Layer (Pub/Sub Message Broker)

To sustain an outbound delivery rate of 5,000,000 messages per second, a Pub/Sub architecture is essential. This is achieved through distributed brokers like Redis, NATS, or Apache Kafka.

- **Operational Mechanics**: When a message is sent, the designated WebSocket gateway publishes the frame to a centralized channel_id topic within the broker.
- **Smart Broadcasting**: Each gateway subscribes only to the active channels relevant to its connected users. For a high-traffic stream, the broker transmits only a single copy of a message to each gateway, which then handles the local fanout to its sockets, preventing internal network saturation.

C. The Database Strategy

Attempting to write 50,000 inbound messages per second directly to a database would create a significant bottleneck. We must decouple real-time delivery from long-term storage.

- **Hot vs. Cold Path Separation**: Real-time message propagation is treated as a latency-sensitive *in-memory* operation.
- **Message Queuing**: Inbound traffic is routed to ingestion queues, such as Apache Kafka, to ensure durability without blocking the main signal path.
- **Batch Database Writes**: Background workers consume these queues and perform bulk writes to highly scalable NoSQL databases like Apache Cassandra for persisted chat history.

In summary, this architectural overview provides a comprehensive look at designing a scalable live streaming and video-on-demand platform. The discussion details everything from core system requirements to deep dives into platform functionality, demonstrating how to successfully achieve low latency, navigate CAP theorem trade-offs, and implement robust horizontal scaling.

References

- **System Design Whiteboard:** Excalidraw Link
- **System Design for Beginners Course (freeCodeCamp):** Watch on YouTube
- **Design a Video Streaming Platform Like YouTube (Hello Interview):** View Article
- **Design YouTube - System Design Interview (NeetCode):** Watch on YouTube